YouTube → Faisal Memon

Collections

Collection: group of objects

Collection Framework is a toolbox for managing group of objects

- List
- Queue
- Map
- Set

Array

- Fixed size
- Same data Type
- Lacks ready made methods for common operations

Collection

- Dynamic size
- Heterogeneous data Type
- Comes with ready made methods for common operations

⊙ List:-

List is sub-interface of the Java Collection Framework that represents on ordered collection of elements.
it allows duplicate elements and provides index-based access to elements.

✱ Key Features of List

1. Ordered Collection
2. Index-based
3. Allows duplicates
4. Dynamic resizing
5. Implements Collection interface.

✱ Common Implementing Classes

1. ArrayList
2. LinkedList
3. Vector
4. Stack.

Common Methods of List

1. add (E e)
2. add (int index, E e) → element
3. get (int index)
4. set (int index, E e)
5. remove (int index)
6. indexOf (Object o)
7. lastIndexOf (Object o)

How-work: "Work on all Methods"

Example →

```
import Java.util.ArrayList;
import Java.util.List;

Public class ListDemo {
    Public static void main(String[] args){
    List users = new ArrayList();
        users.add ("Alice");
        users.add ("Bob");
        users.add ("Charlie");
        users.add (1) — // other data type
        users.add ("Alice"); → duplicate
```

```
System.out.println("All Users");
for (Object user : users) {
    System.out.println(user);
}
// Index:
System.out.println("Element using index" +
        users.get(0));
}
```

Output:
All Users
Alice
Bob
charlie

Note: In this "Example" - there is no type safety - any type of data can be added."

Example-2 List of Object:

```
import java.util.ArrayList;
import java.util.List;

class Car {
    String brand;
    Car(String brand) {
        this.brand = brand
    }
}
```

Parameter Pass

```
Public class ListDemo {
    Public static Void main(String[] args) {
List<String> users = new ArrayList<>();

users.add("Alice");
users.add("Bob");
users add ("charlie");
users.add("Alice");

System.out.println("All Users");

  for (String user : users) {
        System.out.println(user);

  }
System.out.println("Element using index"
      + users.get(0));

// List OF OBJECTS
    Car car1 =new Car(brand: "Toyota");
    Car car2 = new Car(brand: "Ford");
List<Car> carList = new ArrayList<X();
  CarList.add(car1);
  carList.add (car2);
System.out.println ("All Cars");
```

// for (String user : Users) ⇒ Not allow

```
For (Car car : carList) {
    System.out.println(car.brand);
}
}
```

Output: Only ~~Method~~ Object output show not any show in example → you see → alia, User. name

All Cars
Toyota
Ford.

⊙ SET:

Set is a part of the Java Collection Framework the represents a collection of unique elements

It does not allow duplicate elements and does not maintain insertion Order (except in some implementations).

✡ Key Features of set

1. No duplicates allowed
2. Unordered collection
3. Null value
4. Implements the collection interface

✡ Common Implementing Classes

1. HashSet
2. LinkedHashSet
3. TreeSet

✡ Common Methods of Set Interface:

1. add(E e)
2. remove(Object o)
3. Contains(Object o)
4. size()
5. clear()
6. isEmpty()

Example

```
import java.util.HashSet;
import java.util.Set;

Public class SetDemo{
  Public static void main(String[] args){
   Set<String>roles = new HashSet<>();
   roles.add("ADMIN");
   roles.add("USER");
   roles.add("MANAGER");

   For(String role :roles)
    System.out.println("ROLE:"+role);
  }
}
```

Output:

USER
ADMIN
MANAGER
(again)

→ ADMIN
USER
MANAGER

u → Not follow Indexing

10. Map

Map is an object that maps keys to values. It is part of the Java Collection Framework but does not extend the collection interface.

* Each key in a Map is unique, but values can be duplicate.

Key Features of Map

1. Stores data in key-value pairs
2. Keys must be unique
3. Values can be duplicate
4. Does not maintain order (unless using LinkedHashMap or TreeMap).
5. Useful for fast lookups based on key
6. Commonly used when you want to associate a value with key

Common Implementing Classes

1. HashMap
2. LinkedHashMap
3. TreeMap
4. Hashtable

✱ Common Methods of Map

1. Put(K key, V value)
2. get(Object key)
3. remove(Object key)
4. ContainsKey(Object key)
5. ContainsValue(Object value)
6. KeySet()
7. values()
8. entrySet()

Example

```
import java.util.HashMap;
import java.util.Map;
Public class MapDemo {
  Public Static Void main(String
   [] args) {
  // Creating a Map with Integer
   Keys and String Values:
Map<Integer, String> UserMap =
   new HashMap<>();

// Adding Key Value pairs
```

```
userMap.put(1, "Alice");
userMap.put(2, "Bob");
userMap.put(3, "Charlie");

// Fetching value by key
System.out.println("User with id 2:" +
                   userMap.get(2));
    }
}
```

Output: User with id 2: Bob

Generics in Java

Generics allow classes, Interfaces and methods to operate on any data type while providing type safety and reusability.

Example

```
import java.util.ArrayList;
import java.util.List;

// Simple Car class
class CarNew {
    String brand;
```

```
CarNew(String brand) {
    this.brand = brand;
  }
}
Public class GenericsDemo {
    Public static void main(String [] 
    args) {
// List OF OBJECTS USING GENERICS
    CarNew Car1 = new CarNew("Toyota");
      CarNew car2 = new CarNew("Ford");
    List<CarNew> carList = new ArrayList
                          <>(); //Generics
      CarList.add(car1);
      Carlist.add(car2);
// Print all cars using for-each
    loop
System.out.println("== All Cars ==");
      for (CarNew car : carList) {
  System.out.println(car.brand);
      }
```

// Accessing element using index

System.out.println("Element at index 1 : " + carList.get(1).brand);

}

}

Example- Explanation:

(1) CarNew class का एक Property brand है।

(2) List<CarNew> का use करके हम type safe list बनाते है।

(3) for-each loop से सभी cars Print करते है

(4) carList.get(index) से किसी specific index का element लिया जाता है।

✡ IteratingDemo.Java.

```
import java.util.ArrayList;
import Java.util.List;
Public class IteratingDemo {
    Public static void main(String[]
    args) {
```

```
List<String> users = new ArrayList<>();
users.add("Alice");
users.add("Bob");
users.add("Charlie");
users.add("Alica");

// For each
System.out.println("USING For EACH");
for (String user : users)
    System.out.println(user);

// For Loop
System.out.println("USING FOR LOOP");
for (int i=0; i< users.size(); i++){
System.out.println(users.get(i));

// ITERATOR (-
System.out.println("USING for ITERATOR");
Iterator<String> it = users.iterator();
while (it.hasNext())
System.out.println(it.next());
    if (it.next().equals("Alica")){
        it.remove.
}
}
}
```

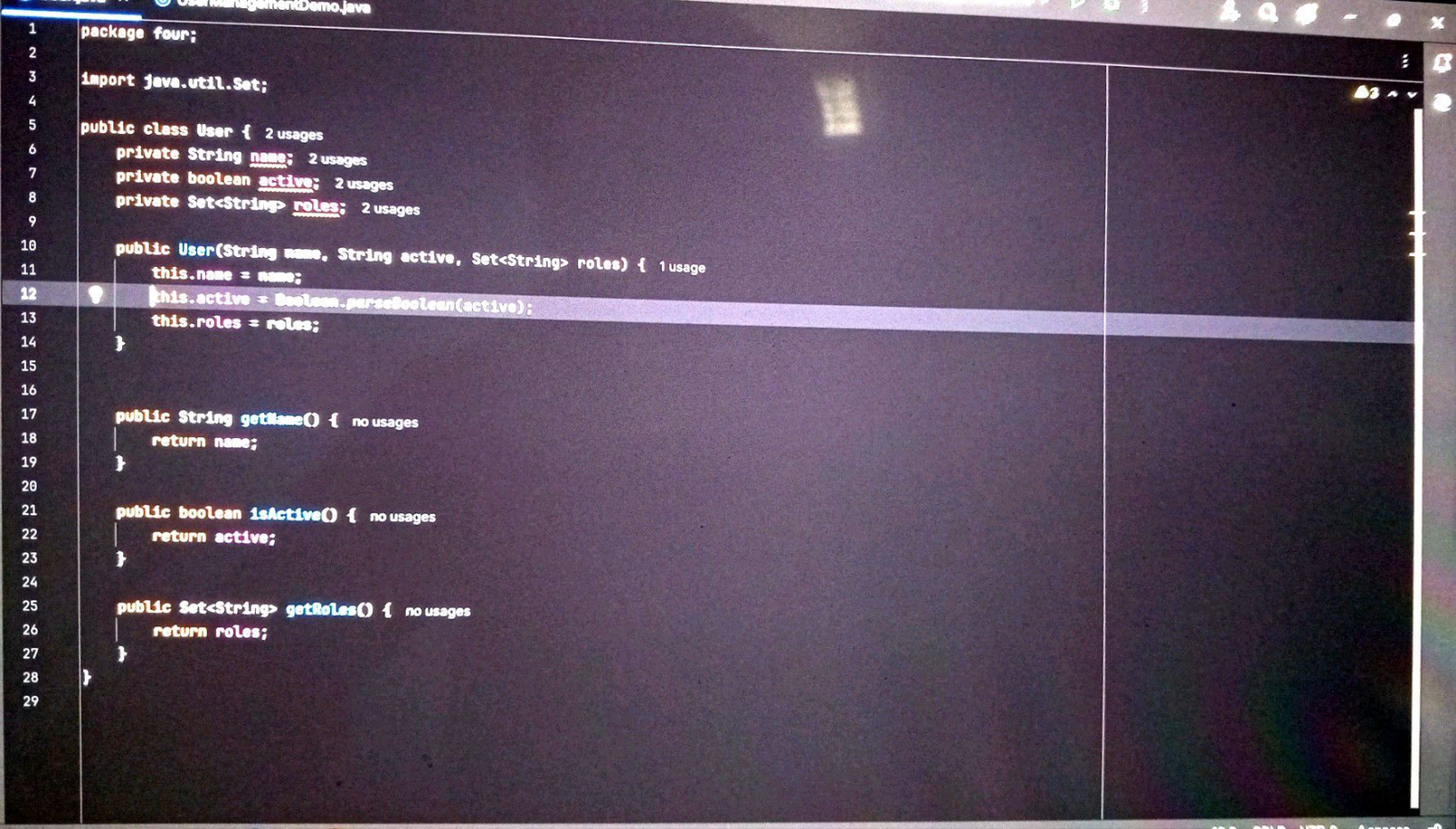
PROJECT

Build a system that:

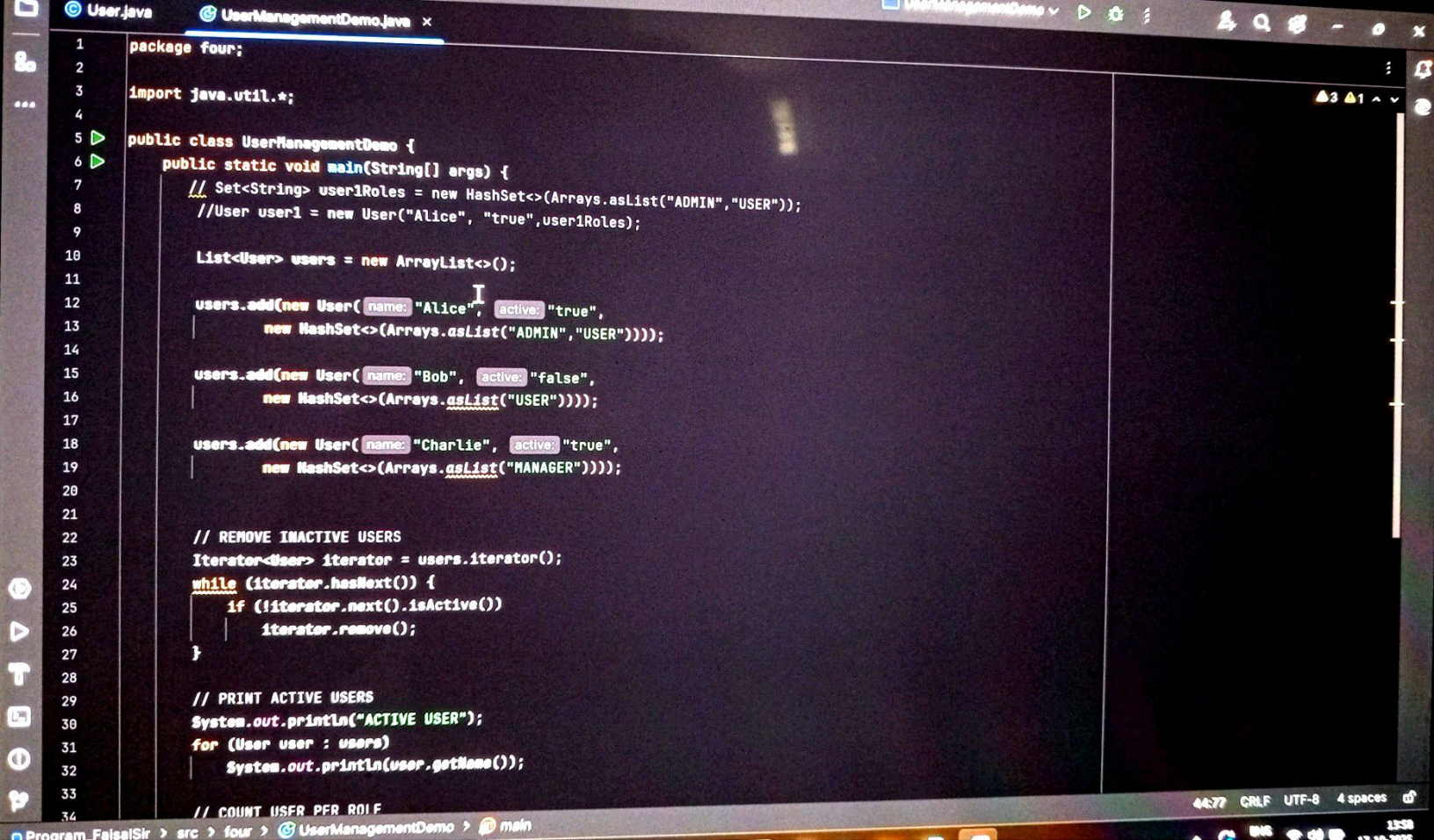
- Fetches users from a database (we'll mock it):
- Each user has roles (unique).

- We want to:
- Remove inactive users
- List all active users
- Count users per Role

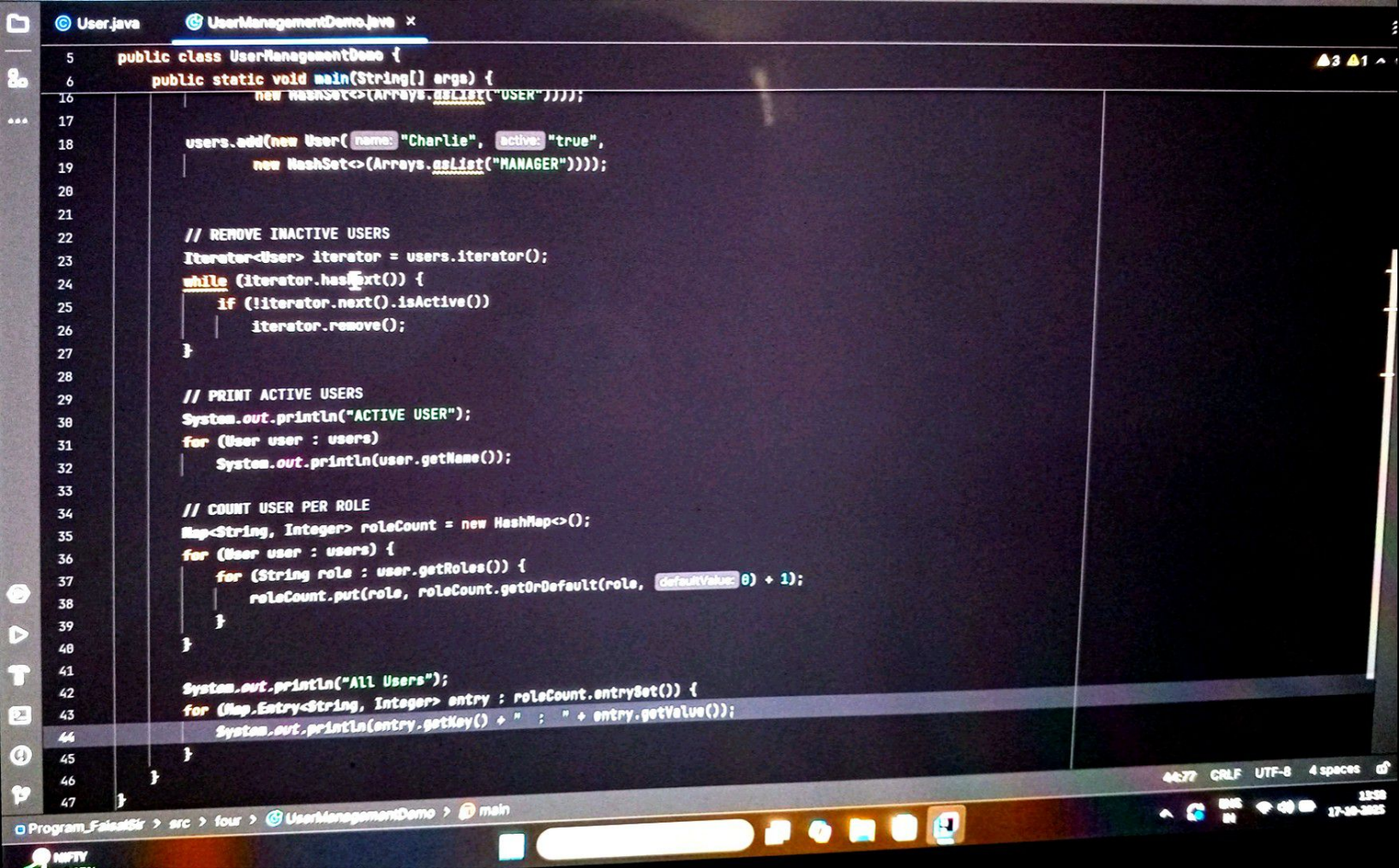
→ User.java

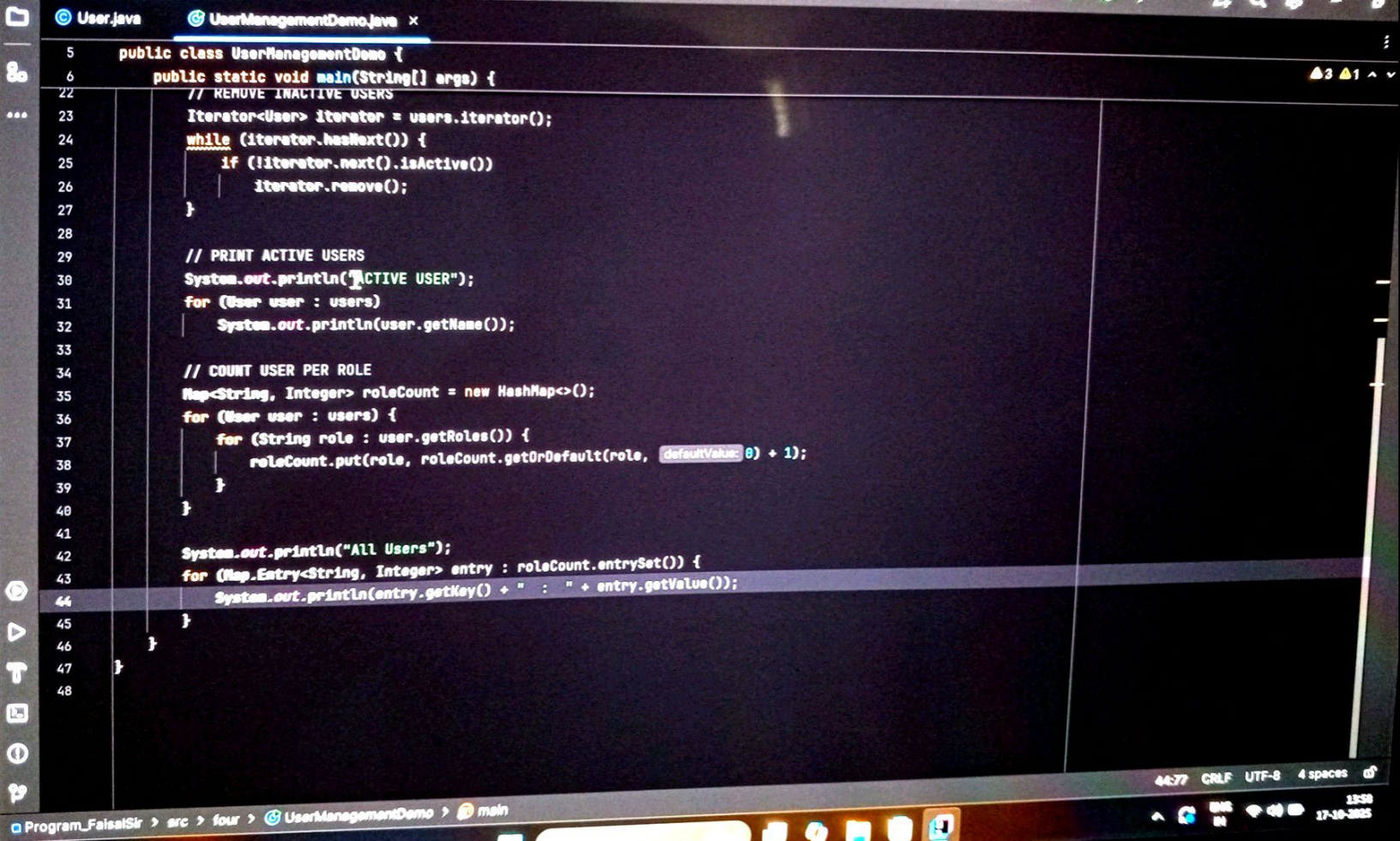


```java
package four;

import java.util.Set;

public class User {  2 usages
    private String name;  2 usages
    private boolean active;  2 usages
    private Set<String> roles;  2 usages

    public User(String name, String active, Set<String> roles) {  1 usage
        this.name = name;
        this.active = Boolean.parseBoolean(active);
        this.roles = roles;
    }


    public String getName() {  no usages
        return name;
    }

    public boolean isActive() {  no usages
        return active;
    }

    public Set<String> getRoles() {  no usages
        return roles;
    }
}

```



```java
package four;

import java.util.*;

public class UserManagementDemo {
    public static void main(String[] args) {
        // Set<String> user1Roles = new HashSet<>(Arrays.asList("ADMIN","USER"));
        //User user1 = new User("Alice", "true",user1Roles);

        List<User> users = new ArrayList<>();

        users.add(new User( name: "Alice",  active: "true",
                new HashSet<>(Arrays.asList("ADMIN","USER"))));

        users.add(new User( name: "Bob",  active: "false",
                new HashSet<>(Arrays.asList("USER"))));

        users.add(new User( name: "Charlie",  active: "true",
                new HashSet<>(Arrays.asList("MANAGER"))));


        // REMOVE INACTIVE USERS
        Iterator<User> iterator = users.iterator();
        while (iterator.hasNext()) {
            if (!iterator.next().isActive())
                iterator.remove();
        }

        // PRINT ACTIVE USERS
        System.out.println("ACTIVE USER");
        for (User user : users)
            System.out.println(user.getName());

        // COUNT USER PER ROLE
```


User.java
UserManagementDemo.java
public class UserManagementDemo {
public static void main(String[] args) {
new HashSet<>(Arrays.asList("USER"))));
users.add(new User(name: "Charlie", active: "true",
new HashSet<>(Arrays.asList("MANAGER"))));
// REMOVE INACTIVE USERS
Iterator<User> iterator = users.iterator();
while (iterator.hasNext()) {
if (!iterator.next().isActive())
iterator.remove();
}
// PRINT ACTIVE USERS
System.out.println("ACTIVE USER");
for (User user : users)
System.out.println(user.getName());
// COUNT USER PER ROLE
Map<String, Integer> roleCount = new HashMap<>();
for (User user : users) {
for (String role : user.getRoles()) {
roleCount.put(role, roleCount.getOrDefault(role, defaultValue: 0) + 1);
}
}
System.out.println("All Users");
for (Map.Entry<String, Integer> entry : roleCount.entrySet()) {
System.out.println(entry.getKey() + " : " + entry.getValue());
}
}
}
44:77
CRLF
UTF-8
4 spaces
Program_FaisalBir > src > four > UserManagementDemo > main


User.java
UserManagementDemo.java
public class UserManagementDemo {
public static void main(String[] args) {
// REMOVE INACTIVE USERS
Iterator<User> iterator = users.iterator();
while (iterator.hasNext()) {
if (!iterator.next().isActive())
iterator.remove();
}
// PRINT ACTIVE USERS
System.out.println("ACTIVE USER");
for (User user : users)
System.out.println(user.getName());
// COUNT USER PER ROLE
Map<String, Integer> roleCount = new HashMap<>();
for (User user : users) {
for (String role : user.getRoles()) {
roleCount.put(role, roleCount.getOrDefault(role, defaultValue: 0) + 1);
}
}
System.out.println("All Users");
for (Map.Entry<String, Integer> entry : roleCount.entrySet()) {
System.out.println(entry.getKey() + " : " + entry.getValue());
}
}
}
44:77
CRLF
UTF-8
4 spaces
Program_FaisalSir
src
four
UserManagementDemo
main

```
43          for (Map.Entry<String, Integer> entry : roleCount.entrySet()) {
44              System.out.println(entry.getKey() + "  :  " + entry.getValue());
45          }
```

Run UserManagementDemo ×

```
C:\Users\sandy\.jdks\openjdk-24.0.1\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2024
ACTIVE USER
Alice
Charlie
All Users
MANAGER  :  1
ADMIN  :  1
USER  :  1

Process finished with exit code 0
```